

DD: FUL-02-STEP-PAYMENT — Payment Capture

Developer implementation guide: DD_FUL-02-STEP-PAYMENT-DEV_IMPL_GUIDE-v1.0.md.

1. Purpose

This rewritten DD is the developer-facing version of the module contract DD_FUL-02-STEP-PAYMENT-v1.0.md.

Verdict: ■ New | **Wave:** 1 | **Group III:** Fulfillment | **Version:** v1.0 | **Satellite of:** DD_FUL-02-Order_Capture-v1.0.md
Computes the upfront amount the customer needs to pay, tells them how to pay it, and waits for the payment to arrive before letting the order advance. The "pay-first" gate. Phase: AWAITING_PAYMENT.

The goal of this rewrite is to make the DD easier for a mid-level developer to implement without losing the detailed source contract.

2. Implementation Scope

This DD should be implemented as part of the owning SOPHIX capability, not as an isolated service unless the deployment architecture explicitly separates that capability.

Implement this DD with these rules:

- Use the owning module APIs/repositories for authoritative writes.
- Use approved internal APIs/client wrappers when reading another module's data.
- Do not query another module's database tables directly from business flow code.
- Treat worker names as logical Camunda external-task topics, not separate deployments.
- One worker application may handle multiple topics, but metrics, retries, and logs must remain visible per topic.
- Emit success events only after the authoritative state commit succeeds.
- Use outbox publishing for Kafka events.

3. Runtime Metadata

Item	Value
Source file	/Users/macbook/Downloads/Sophix_V3_BSS_DDs_MD_2026-05-27/06_fulfillment/DD_FUL-02-STEP-PAYMENT-v1.0.md
Version	v1.0
DD type	module contract
BPMN/process keys	Not explicitly defined
Drools packages	Not explicitly defined

4. Runtime Contracts To Implement

4.1 APIs

- GET /api/billable-events/by-trigger/ACTIVATION_INTENT?operatorCode={op}
- GET /api/discounts/applicable?packageRef={ref}&customerId={id}
- GET /api/services/{ref}
- POST /api/admin/orders/{id}/mark-paid-offline
- POST /api/payments/receive
- POST /engine-rest/message

4.2 Tables

- No CREATE TABLE block is explicitly declared in this DD.

For each table in this DD, implement the schema with:

- a short table comment explaining what the table is for;
- column comments or migration notes explaining each field;
- constraints and indexes from the DD;
- seed data where the DD provides operator-specific sample rows.

4.3 Worker Topics

- compute-expected-payment
- create-subscription
- kyc-l1-supervisor-approval
- notify-payment-instructions
- order-cascade-cancellation
- order-compute-expected-payment
- order-mark-cancelled
- order-notify-payment-instructions
- payment-received-partial
- wait-payment

Worker-topic guidance:

- A BPMN service task uses the topic.
- A worker application subscribes to the topic.
- The topic handler validates input variables, calls the owning module/API, writes outputs back to Camunda, and records metrics.
- Do not treat the topic string as a shell command or Kubernetes deployment name.

4.4 Events

- PaymentApplied
- PaymentAppliedConsumer
- PaymentReceived

Event guidance:

- Write events through the transactional outbox.
- Include operation id, aggregate id, operator code, actor, and correlation data.
- Consumers should be able to rebuild downstream state without querying workflow internals.

5. Developer Implementation Checklist

1. Add or verify database migrations and seed rows.
2. Add or verify config rows for the operator/module.
3. Implement request/command DTOs and validation.
4. Implement idempotency and in-flight operation checks where the DD requires them.
5. Implement Drools fact mapping and package deployment where rules are used.
6. Implement BPMN process, service tasks, gateways, timers, and message catches where the DD is a flow.
7. Implement worker-topic handlers using owning module APIs.
8. Implement compensation paths and manual recovery paths.
9. Emit success/rejected events through the outbox.
10. Add smoke tests for happy path, validation failure, dependency failure, and retry/idempotency.

6. Infrastructure And AWS Notes

Deploy this DD with the existing SOPHIX platform components unless the owning module requires isolation:

Concern	Recommendation
API/runtime	Run inside the owning module deployment or existing workflow API pods.
Workers	Consolidate low-volume topics into shared worker apps; keep per-topic metrics.
Database	Use the owning module PostgreSQL schema and migration pipeline.
Kafka	Use the foundation Kafka/MSK setup and transactional outbox.
Camunda	Deploy BPMN files through the workflow deployment pipeline.
Drools	Deploy KJAR/rule artifacts through the approved rules pipeline.
Secrets	Use the platform secret manager; on AWS prefer Secrets Manager/Parameter Store with IRSA.
Observability	Alert on stuck operations, failed workers, outbox backlog, and external dependency failures.

7. Original DD Detail Preserved

The following section preserves the detailed source contract for implementation and review.

Verdict: ■ New | **Wave:** 1 | **Group III:** Fulfillment | **Version:** v1.0 | **Satellite of:** DD_FUL-02-Order_Capture-v1.0.md

Computes the upfront amount the customer needs to pay, tells them how to pay it, and waits for the payment to arrive before letting the order advance. The "pay-first" gate. Phase: AWAITING_PAYMENT.

1. What this step does

In KE WIK, the operator policy is **pay-first**: the customer must pay the installation fee and first cycle prorata before any tech is dispatched. The order can't advance past this gate until the customer's payment lands in BIL-01.

This step has three moving parts:

- 1. Compute the expected amount** (order-compute-expected-payment worker). The amount depends on the Package and the activation date. The worker resolves it by walking the catalogs: BIL-CFG-01 tells us which BillableEvents fire on ACTIVATION_INTENT (typically INSTALLATION_FEE + FIRST_CYCLE_PRORATE); each BillableEvent's service_refs[] resolves to one-shot Services in PLM-CFG-01 carrying prices; PLM-CFG-04 may apply discounts. The worker sums those into a single expectedPaymentAmount, writes it onto customer_order, sets a Camunda process variable, completes.
- 2. Tell the customer how to pay** (order-notify-payment-instructions worker). Emits the OrderAwaitingPayment Kafka event. DD_NOT-01 consumes that event and sends the customer an SMS+email with the paybill instructions (M-Pesa for WIK). The BPMN then parks at a **message catch event** named wait-payment — a BPMN step that sits and waits until someone notifies Camunda with messageName = PaymentReceived — guarded by a **boundary timer** (PT7D for WIK). The boundary timer is a side-step attached to the wait that fires after the configured duration if no message arrives — used to enforce a timeout. If the customer pays in time, the message fires and the BPMN advances. If the timer fires first, the order goes to STEP-CANCELLATION with reason PAYMENT_TIMEOUT.
- 3. React to payment arriving** (PaymentAppliedConsumer Kafka consumer). When the customer pays, the payment gateway adapter calls BIL-01's POST /api/payments/receive. BIL-01 allocates the payment (credit balance for POSTPAID customers in CREATED-subscription state, since no invoice exists yet) and emits sophix.billing.payment.applied. Our consumer picks up that event, looks up which order it's for, checks whether the cumulative amount is enough, and either notifies Camunda's wait-payment to wake up (full payment) or just updates the order with the partial amount (more is needed).

Correlating payments back to orders

The PaymentApplied event does NOT carry orderId directly — BIL-01 doesn't know about FUL-02's orders. It carries payment_reference (the gateway-generated reference, like mpesa_NK4XX7QP9) and targetReference (the reference the customer typed when paying, like the M-Pesa account number). Our consumer needs to map either of those back to an order.

The mechanism: when `order-notify-payment-instructions` tells the customer to pay, it tells them to type `order_id` as the M-Pesa account number (or equivalent for the operator's gateway). The customer types it; the gateway captures it as `accountNumber`; BIL-01 echoes it as `targetReference` in the `PaymentApplied` event payload. To make this lookup fast and explicit, the worker writes a side-channel row to `customer_order_pending_payments` keyed by `payment_reference = order_id` at notify time. `PaymentAppliedConsumer` then correlates via `event.targetReference → customer_order_pending_payments.payment_reference → order_id`.

This side-channel exists because the alternative (looking up via `subscription_id → invoice → payment`) doesn't work in pay-first KE: at payment time, no invoice has been issued yet (the activation saga hasn't charged anything). The payment lands on the customer's credit balance, and there's no invoice-side handle. The side-channel keyed on `order_id` is the cleanest correlation.

Partial payment

Customers can split payments — pay 10000 KES today, 5000 KES tomorrow. Each `PaymentApplied` event is processed independently. The consumer accumulates the cumulative paid amount onto `customer_order.payment_received_amount`. If the cumulative is $\geq$ `expected_payment_amount`, the consumer fires `PaymentReceived` to Camunda; the BPMN advances. If it's below the threshold, the consumer emits a partial-payment Kafka event (consumed by DD_NOT-01 to tell the customer "we got X of Y; please pay the remaining Z") and the BPMN stays parked.

What this step deliberately doesn't do

- **Doesn't charge the customer.** Charging happens in STEP-ACTIVATION's activation saga, where BIL-01 charges the `BillableEvents` against the credit balance. This step just collects the upfront top-up. The actual amounts charged at activation may differ slightly (BIL-01 reapplies effective discounts, taxes, etc.).
- **Doesn't dispatch the customer payment via push.** We tell them the paybill; they push the payment from their side (M-Pesa STK push is initiated by the customer, not us). For card-on-file scenarios (out of v1.0 scope), the model would be different.
- **Doesn't enforce currency conversion.** The expected amount is in KES for KE WIK. Cross-currency is out of scope.

2. Worked example end-to-end (WIK, single full payment)

Continuing the John Mwangi scenario from STEP-CAPTURE and STEP-SUBSCRIPTION. Order `ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1`, customer `cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6`, package `pkg_INET_VOIP_RES`, subscription `sub_01HX5M2K9P3Q7R4T6W8Y0Z2N4Q`. Camunda has just advanced past `create-subscription` to `compute-expected-payment`.

Step 1 — Worker computes the expected amount

`ComputeExpectedPaymentWorker` picks up the task. Process variables on the task: `orderId`, `customerId`, `packageRef = pkg_INET_VOIP_RES`, `homepassId`, `operatorCode = WIK`, `subscriptionId = sub_01HX...A`, `intendedActivationDate = null` (immediate).

The worker walks the catalog to compute the activation cost:

1. Fetch active `BillableEvents` for `ACTIVATION_INTENT`, operator `WIK`, package `pkg_INET_VOIP_RES`:
 - `INSTALLATION_FEE` (`service_refs: [svc_installation_residential_gpon]`)
 - `FIRST_CYCLE_PRORATE` (`service_refs: [svc_first_cycle_prorate]`)
(filter: `cycleAnchorOverflowApplied == PRORATE_PARTIAL_CYCLE` — for now, immediate activation on day 18, anchor day TBD by `SUB-WF-ACTIVATE-01`; assume operator's default overflow policy resolves to anchor = today-day = 18, no proration needed)
2. Resolve each one-shot Service's price from `PLM-CFG-01`:
 - `svc_installation_residential_gpon` → price 5000.00 KES (catalog price)
 - `svc_first_cycle_prorate` → price computed from Package's monthly cycle price × prorate ratio
Package `pkg_INET_VOIP_RES` monthly price = 10000.00 KES
For immediate activation with anchor = today: full first cycle
→ 10000.00 KES
3. Apply `PLM-CFG-04` discounts if any active for this Package + customer cohort:
No active discounts for this customer → no adjustment.
4. Sum: 5000.00 + 10000.00 = 15000.00 KES.

The exact prorate logic depends on `cycle_anchor_day` and the operator's overflow policy, which is officially resolved by `SUB-WF-ACTIVATE-01` at activation time (per `R-SUB-LM-01-S-6a`). `STEP-PAYMENT` uses a **best-estimate** prorate

calculation here — the customer-facing quote. The actual amount charged during activation may differ by a small amount due to rounding or operator policy; any difference is reconciled by BIL-01 (overpayment → wallet credit; underpayment → next invoice). For v1.0, we accept this minor estimation gap.

Worker UPDATES the order:

```
UPDATE customer_order
  SET expected_payment_amount = 15000.00,
      expected_payment_currency = 'KES',
      last_updated_at = NOW()
 WHERE order_id = 'ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1';
```

Worker completes the task with `expectedPaymentAmount` + `expectedPaymentCurrency` process variables.

Step 2 — Worker notifies the customer how to pay

Camunda advances to `notify-payment-instructions`. The `NotifyPaymentInstructionsWorker` picks it up.

First, it writes a row to `customer_order_pending_payments` so `PaymentAppliedConsumer` can later correlate:

```
INSERT INTO customer_order_pending_payments (
  order_id, payment_reference, customer_id, operator_code,
  expected_amount, currency, awaiting_since
) VALUES (
  'ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1',
  'ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1', -- = payment_reference from STEP-CAPTURE
  'cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6',
  'WIK',
  15000.00,
  'KES',
  NOW()
);
```

Then emits the Kafka event:

```
{
  "eventType": "OrderAwaitingPayment",
  "aggregateId": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
  "timestamp": "2026-05-18T09:00:03.567Z",
  "payload": {
    "orderId": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
    "customerId": "cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6",
    "operatorCode": "WIK",
    "subscriptionId": "sub_01HX5M2K9P3Q7R4T6W8Y0Z2N4Q",
    "expectedAmount": 15000.00,
    "currency": "KES",
    "paymentReference": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
    "paybillNumber": "555888",
    "paymentMethods": ["MPESA", "AIRTEL_MONEY", "BANK_TRANSFER"],
    "paymentTimeoutAt": "2026-05-25T09:00:03Z"
  }
}
```

DD_NOT-01 consumes this event and sends John Mwangi an SMS:

Hi John, your Zuku Fiber 100M order is ready. Please pay 15000 KES via M-Pesa to paybill 555888, account number `ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1`. Payment must be received by 25 May 2026 — after that the order will be cancelled.

Worker completes. Camunda advances to `wait-payment` — a message catch event with a PT7D boundary timer attached.

PhaseProjector sees `wait-payment` and updates:

```
UPDATE customer_order
  SET current_phase = 'AWAITING_PAYMENT',
      current_activity_id = 'wait-payment',
      last_updated_at = NOW()
 WHERE order_id = 'ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1';
```

The order is now parked. Camunda will sit on `wait-payment` until either `PaymentReceived` arrives via `POST /engine-rest/message` or 7 days pass and the timer fires.

Step 3 — Customer pays at 11:23

John walks to his M-Pesa menu, sends 15000 KES to paybill 555888 with account `ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1`. M-Pesa confirms; M-Pesa's webhook delivers to our payment gateway adapter; adapter calls BIL-01:

```
POST /api/payments/receive HTTP/1.1
Host: bil-01-pay-01.sophix.internal
Content-Type: application/json

{
```

```

"customerId": "cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6",
"operatorCode": "WIK",
"paidAmount": 15000.00,
"currency": "KES",
"paymentMethod": "MPESA",
"paymentReference": "mpesa_NK4XX7QP9_20260518T112301",
"paidAt": "2026-05-18T11:23:01Z",
"targetReference": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1"
}

```

targetReference is what John typed as the M-Pesa account number — the order ID. BIL-01 resolves this to the customer's POSTPAID setup (WIK), there's no invoice yet (subscription is in CREATED), so it applies the payment to the customer's credit balance (operator's policy APPLY_TO_NEXT_OPEN for an unallocated payment — the credit will be consumed at activation when BIL-01 charges INSTALLATION_FEE + FIRST_CYCLE).

BIL-01 emits `sophix.billing.payment.applied`:

```

{
  "eventType": "PaymentApplied",
  "aggregateId": "pmt_01HZ_MPESA_NK4XX7QP9",
  "timestamp": "2026-05-18T11:23:02.456Z",
  "payload": {
    "paymentId": "pmt_01HZ_MPESA_NK4XX7QP9",
    "paymentReference": "mpesa_NK4XX7QP9_20260518T112301",
    "customerId": "cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6",
    "operatorCode": "WIK",
    "target": {
      "type": "CREDIT_BALANCE",
      "appliedAmount": 15000.00,
      "balanceAfter": 15000.00
    },
    "totalPaidAmount": 15000.00,
    "currency": "KES",
    "paymentMethod": "MPESA",
    "targetReference": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
    "paidAt": "2026-05-18T11:23:01Z",
    "appliedAt": "2026-05-18T11:23:02Z"
  }
}

```

Step 4 — Our consumer wakes the parked Camunda instance

PaymentAppliedConsumer (Java Kafka consumer in our module) receives the event. Logic:

1. Check `payload.targetReference` (ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1) against `customer_order_pending_payments`:

```

SELECT order_id, expected_amount, currency
FROM customer_order_pending_payments
WHERE payment_reference = 'ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1'
AND resolved_at IS NULL;

```

→ matches order ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1, expected 15000.00 KES.

2. Idempotency: check `customer_order_processed_payments` for (order_id, payment_id):

```

SELECT 1 FROM customer_order_processed_payments
WHERE order_id = 'ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1'
AND payment_id = 'pmt_01HZ_MPESA_NK4XX7QP9';

```

→ no match (first time seen). Insert the row.

3. Update cumulative paid amount:

```

UPDATE customer_order
SET payment_received_amount = payment_received_amount + 15000.00,
    last_updated_at = NOW()
WHERE order_id = 'ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1';

```

Cumulative = 15000.00.

4. Threshold check: cumulative 15000.00 ≥ expected 15000.00 → full payment received. Mark the pending-payment row resolved:

```

UPDATE customer_order_pending_payments
SET resolved_at = NOW(),
    resolved_via = 'pmt_01HZ_MPESA_NK4XX7QP9'
WHERE order_id = 'ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1';

```

5. Wake Camunda via message correlation:

```

POST /engine-rest/message HTTP/1.1

```

```
Host: camunda.sophix.internal
Content-Type: application/json
```

```
{
  "messageName": "PaymentReceived",
  "businessKey": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
  "processVariables": {
    "paymentId": { "value": "pmt_01HZ_MPESA_NK4XX7QP9", "type": "String" },
    "paidAmount": { "value": 15000.00, "type": "Double" },
    "paidCurrency": { "value": "KES", "type": "String" },
    "paymentMethod": { "value": "MPESA", "type": "String" }
  }
}
```

Camunda finds the parked instance with matching businessKey, sees the wait - payment message catch matches the message name, wakes the instance. The BPMN advances past wait - payment toward the next step (in WIK that's STEP-KYC's first user task).

6. Emit sophix.orders.payment-received:

```
{
  "eventType": "OrderPaymentReceived",
  "aggregateId": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
  "timestamp": "2026-05-18T11:23:02.789Z",
  "payload": {
    "orderId": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
    "customerId": "cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6",
    "operatorCode": "WIK",
    "subscriptionId": "sub_01HX5M2K9P3Q7R4T6W8Y0Z2N4Q",
    "paymentId": "pmt_01HZ_MPESA_NK4XX7QP9",
    "paidAmount": 15000.00,
    "currency": "KES",
    "method": "MPESA",
    "cumulativePaid": 15000.00,
    "expectedAmount": 15000.00,
    "receivedAt": "2026-05-18T11:23:01Z"
  }
}
```

DD_NOT-01 consumes this and sends John a confirmation SMS: "Payment of 15000 KES received. Your installation will be scheduled shortly."

Total elapsed in step: ~2h 23min (mostly customer-side: him walking to M-Pesa and paying).

3. Worked example — partial payment

Same scenario but John only sends 10000 KES initially, then tops up 5000 KES later.

Variant of Step 3a — First payment 10000 KES

BIL-01 emits PaymentApplied with appliedAmount = 10000.00. Our consumer:

- Cumulative paid: $0 + 10000 = 10000$.
- Threshold check: $10000 < 15000 \rightarrow$ partial. Don't wake Camunda. Don't resolve the pending-payments row (still awaiting more).
- Emit sophix.orders.payment-received-partial:

```
{
  "eventType": "OrderPaymentReceivedPartial",
  "aggregateId": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
  "timestamp": "2026-05-18T11:23:02.789Z",
  "payload": {
    "orderId": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
    "customerId": "cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6",
    "operatorCode": "WIK",
    "paymentId": "pmt_01HZ_PARTIAL_1",
    "paidAmount": 10000.00,
    "currency": "KES",
    "method": "MPESA",
    "cumulativePaid": 10000.00,
    "expectedAmount": 15000.00,
    "shortfallAmount": 5000.00,
    "receivedAt": "2026-05-18T11:23:01Z"
  }
}
```


DD_NOT-01 sends: "We received 10000 KES. Please pay the remaining 5000 KES via M-Pesa paybill 555888 account ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1 to proceed."

The BPMN stays parked at wait - payment. The PT7D boundary timer continues running from its original start (not reset).

Variant of Step 3b — Second payment 5000 KES, two hours later

Another PaymentApplied event arrives with appliedAmount = 5000.00, paymentId = pmt_01HZ_PARTIAL_2. Consumer:

- Cumulative: 10000 + 5000 = 15000.
- Threshold check: 15000 ≥ 15000 → full. Wake Camunda; resolve pending-payments row; emit OrderPaymentReceived. Same as the happy-path step 4 above, with paymentId = pmt_01HZ_PARTIAL_2 and cumulativePaid = 15000.

4. Worked example — payment timeout

Customer fails to pay within PT7D.

Variant — 7 days after step 2

At 2026-05-25T09:00:03Z Camunda's job executor fires the boundary timer attached to wait - payment. The timer is **interrupting** (cancelActivity="true") — Camunda cancels the wait - payment activity and routes the flow to the timeout branch. The BPMN gateway sees the timeout-side path leads to order - cascade - cancellation with reason PAYMENT_TIMEOUT.

The cascade worker runs (owned by STEP-CANCELLATION), checks what's set on the order:

- subscription_id is set (sub_01HX5M2K9P3Q7R4T6W8Y0Z2N4Q) — calls SUB-WF-TERMINATE-01 to terminate the subscription.
- work_order_id is NULL — no WO to cancel.
- payment_received_amount = 0 — no refund needed.

Cascade emits OrderCancellationCascadeCompleted (see STEP-CANCELLATION). Then order - mark - cancelled finalises:

```
UPDATE customer_order
  SET current_phase = 'CANCELLED',
      failure_reason = 'PAYMENT_TIMEOUT',
      failure_detail = 'Customer did not pay within PT7D of OrderAwaitingPayment',
      last_updated_at = NOW()
 WHERE order_id = 'ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1';
```

Emits OrderCancelled (see STEP-CANCELLATION). DD_NOT-01 consumes it and sends John: "Your order was cancelled because payment was not received within 7 days. Please re-submit if you'd still like to subscribe."

5. Workers

5.1 order-compute-expected-payment

Field	Value
Topic	order-compute-expected-payment
Purpose	Resolve the upfront amount the customer must pay before tech is dispatched. Walks BIL-CFG-01 → PLM-CFG-01 → PLM-CFG-04 to compute.
Process variables read	orderId, packageRef, homepassId, operatorCode, optional intendedActivationDate
External HTTP calls	BIL-CFG-01 GET /api/billable-events/by-trigger/ACTIVATION_INTENT?operatorCode={op} — list of BillableEvents firing at activation. PLM-CFG-01 GET /api/services/{ref} per service in each BillableEvent's service_refs[] — prices. PLM-CFG-04 GET /api/discounts/applicable?packageRef={ref}&customerId={id} — discounts.
Process variables written	expectedPaymentAmount (Double), expectedPaymentCurrency (String)
customer_order columns written	expected_payment_amount, expected_payment_currency

Field	Value
Kafka emissions	On zero-cost packages (rare but valid): sophix.orders.compute-payment-warning for ops verification.
Lock duration	10 000 ms
Retry policy	2 retries with PT1M between attempts (catalogs may be transiently unavailable)

Failure modes:

- **No matching BillableEvents** — possible for zero-cost activation packages (free trials, comp accounts). Worker completes with `expectedPaymentAmount = 0` and emits a warning event for ops verification; the BPMN immediately advances past wait-payment (because cumulative paid $0 \geq$ expected 0). v1.0 operator BPMNs don't enable this path; documented for completeness.
- **PLM-CFG-01 unreachable** → /failure with UPSTREAM_UNREACHABLE. Incident.
- **Service price not found for service_refs[] member** → /failure with CATALOG_INCONSISTENT. Incident — catalog admin must fix.

5.2 order-notify-payment-instructions

Field	Value
Topic	order-notify-payment-instructions
Purpose	Insert the side-channel pending-payment mapping row, emit the OrderAwaitingPayment Kafka event so DD_NOT-01 can SMS the customer.
Process variables read	orderId, customerId, subscriptionId, expectedPaymentAmount, expectedPaymentCurrency, operatorCode. BPMN attribute: paymentTimeoutDuration (PT7D for WIK).
External HTTP calls	None.
Process variables written	None.
customer_order columns written	None — only the side-channel table.
DB tables written	customer_order_pending_payments (INSERT)
Kafka emissions	sophix.orders.awaiting-payment
Lock duration	5 000 ms
Retry policy	3 retries with PT30S between attempts (Kafka transient)

6. Message catch event

6.1 PaymentReceived

A message catch is a BPMN step that parks the process instance until someone notifies Camunda with a matching message name. Used here to wait for payment without polling.

Triggering signal: our own PaymentAppliedConsumer calls Camunda's POST /engine-rest/message with `messageName = PaymentReceived` once the cumulative paid amount on the order reaches the threshold.

Where BPMNs park: `<bpmn:intermediateCatchEvent>` named wait-payment in every operator BPMN that includes a pay-first phase.

Boundary timer: interrupting timer attached. WIK PT7D. On fire → BPMN routes via timeout branch to order-cascade-cancellation with reason PAYMENT_TIMEOUT.

Process variables set on correlation: `paymentId`, `paidAmount`, `paymentMethod`, `paidCurrency` (passed by PaymentAppliedConsumer so downstream BPMN steps can read them).

6.2 PaymentAppliedConsumer

Field	Value
Subscribes to	Kafka topic <code>sophix.billing.payment.applied</code> (emitted by BIL-01-PAY-01)
Purpose	Map BIL-01's PaymentApplied events to FUL-02 orders, accumulate cumulative paid amount, wake parked Camunda instances when threshold reached.

Field	Value
Lookup strategy	Primary: <code>customer_order_pending_payments.payment_reference = event.targetReference</code> (where the customer typed our <code>order_id</code> as the M-Pesa account number). Fallback: <code>customer_order_pending_payments.payment_reference = event.payment_reference</code> (if BIL-01's <code>target_invoice_id</code> resolution chain ended up using a different reference). On no match, log DEBUG and skip — the payment is for some non-FUL-02 concern.
Idempotency	<code>customer_order_processed_payments(order_id, payment_id)</code> unique constraint — at-least-once Kafka delivery means we may see the same <code>payment_id</code> twice. Second processing is skipped.
<code>customer_order</code> columns written	<code>payment_received_amount</code> (cumulative increment)
DB tables written	<code>customer_order_processed_payments</code> (INSERT per processed payment), <code>customer_order_pending_payments</code> (UPDATE <code>resolved_at</code> on full-payment threshold hit)
Process variables written	Via message correlation: <code>paymentId</code> , <code>paidAmount</code> , <code>paymentMethod</code> , <code>paidCurrency</code>
Camunda call	POST <code>/engine-rest/message</code> on full-payment threshold
Kafka emissions	On full: <code>sophix.orders.payment-received</code> . On partial: <code>sophix.orders.payment-received-partial</code> .
DLQ	On no matching order: log DEBUG, don't DLQ (event is for someone else). On Camunda 400 <code>MismatchingMessageCorrelationException</code> (timer already fired, order is CANCELLED): forward original event to <code>sophix.orders.message-correlation-dlq</code> for finance review — they may decide to refund.

7. DB tables touched

customer_order (FRAMEWORK §5.1) — UPDATE

Column	When set	Set by	Example value
<code>expected_payment_amount</code>	After cost computation	<code>order-compute-expected-payment-worker</code>	15000.00
<code>expected_payment_currency</code>	After cost computation	<code>order-compute-expected-payment-worker</code>	KES
<code>payment_received_amount</code>	Cumulative on each payment	<code>PaymentAppliedConsumer</code>	0.00 → 10000.00 → 15000.00
<code>current_phase</code>	On BPMN entry to <code>wait-payment</code>	<code>PhaseProjector</code>	AWAITING_PAYMENT
<code>current_activity_id</code>	On each transition	<code>PhaseProjector</code>	<code>compute-expected-payment</code> → <code>notify-payment-instructions</code> → <code>wait-payment</code>
<code>last_updated_at</code>	On every UPDATE	<code>trigger / consumer / worker</code>	2026-05-18 11:23:02.789

Sample data — five orders at various payment stages:

order_id	operator_code	expected_amount	currency	received_amount	current_phase	current_activity_id
ord_01HZ...A	WIK	15000.00	KES	15000.00	IN_KYC (paid in full, advanced to KYC L1)	kyc-l1-supervisor-approval
ord_01HZ...B	WIK	12000.00	KES	8000.00	AWAITING_PAYMENT (partial; waiting for 4000 more)	wait-payment
ord_01HZ...C	WIK	15000.00	KES	0.00	AWAITING_PAYMENT (not yet paid; PT7D ticking)	wait-payment
ord_01HZ...D	WIK	15000.00	KES	15000.00	IN_KYC (paid in full, advanced to KYC L1)	kyc-l1-supervisor-approval
ord_01HZ...E	WIK	15000.00	KES	0.00	CANCELLED (timer fired; never paid)	end

customer_order_pending_payments — NEW SIDE-CHANNEL TABLE

The side-channel mapping that lets PaymentAppliedConsumer correlate PaymentApplied events back to FUL-02 orders (since the event itself doesn't carry orderId). One row per active order awaiting payment.

Column	Type	Notes
id	TEXT PK	ULID. Example: copp_01HZX9K8M7Q2N4P6R3T5W8Y0Z1
order_id	TEXT NOT NULL	FK to customer_order
payment_reference	TEXT NOT NULL	The reference the customer types when paying. Equals order_id for v1.0. Example: ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1
customer_id	TEXT NOT NULL	Denormalized for fast lookup
operator_code	TEXT NOT NULL	Denormalized
expected_amount	NUMERIC(18,2) NOT NULL	Amount we're awaiting. Example: 15000.00
currency	TEXT NOT NULL	Example: KES
awaiting_since	TIMESTAMP NOT NULL	When the OrderAwaitingPayment event fired. Example: 2026-05-18 09:00:03.567
resolved_at	TIMESTAMP NULL	NULL while awaiting; set when threshold hit (full pay received). Example: 2026-05-18 11:23:02.456
resolved_via	TEXT NULL	The final payment_id that crossed the threshold. Example: pmt_01HZ_MPESA_NK4XX7QP9

Indexes: PK; UNIQUE (payment_reference) partial WHERE resolved_at IS NULL; (order_id); (operator_code, awaiting_since). P5Y retention (audit + dispute resolution).

Sample data:

id	order_id	payment_reference	expected_amount	currency	awaiting_since	resolved_at	resolved_via
copp_...A	ord_01HZ...A	ord_01HZ...A	15000.00	KES	2026-05-18 09:00:03	2026-05-18 11:23:02	pmt_01HZ_MPESA_NK4XX7QP9
copp_...B	ord_01HZ...B	ord_01HZ...B	12000.00	KES	2026-05-18 10:15:11	NULL	NULL
copp_...C	ord_01HZ...C	ord_01HZ...C	15000.00	KES	2026-05-18 12:30:00	NULL	NULL
copp_...D	ord_01HZ...D	ord_01HZ...D	15000.00	KES	2026-05-18 14:00:08	2026-05-18 14:32:11	pmt_01HZ_MPESA_AS66XX

customer_order_processed_payments — NEW IDEMPOTENCY TABLE

At-least-once Kafka delivery means the same PaymentApplied event may arrive twice. This table records which payment_id we've already processed.

Column	Type	Notes
id	BIGSERIAL PK	
order_id	TEXT NOT NULL	FK to customer_order
payment_id	TEXT NOT NULL	From BIL-01's PaymentApplied event
applied_amount	NUMERIC(18,2) NOT NULL	Amount this specific event added to cumulative
processed_at	TIMESTAMP NOT NULL	When our consumer processed it

Indexes: PK; UNIQUE (order_id, payment_id); (order_id, processed_at DESC). P5Y retention.

Sample data:

order_id	payment_id	applied_amount	processed_at
ord_01HZ...A	pmt_01HZ_MPESA_NK4XX7QP9	15000.00	2026-05-18 11:23:02.789
ord_01HZ...B	pmt_01HZ_PARTIAL_1	8000.00	2026-05-18 10:18:33.412
ord_01HZ...D	pmt_01HZ_MPESA_AS66XX	15000.00	2026-05-18 14:32:11.105

8. Kafka events emitted

sophix.orders.awaiting-payment

```
{
  "eventType": "OrderAwaitingPayment",
  "aggregateId": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
  "timestamp": "2026-05-18T09:00:03.567Z",
  "payload": {
    "orderId": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
    "customerId": "cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6",
    "operatorCode": "WIK",
    "subscriptionId": "sub_01HX5M2K9P3Q7R4T6W8Y0Z2N4Q",
    "expectedAmount": 15000.00,
    "currency": "KES",
    "paymentReference": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
    "paybillNumber": "555888",
    "paymentMethods": ["MPESA", "AIRTEL_MONEY", "BANK_TRANSFER"],
    "paymentTimeoutAt": "2026-05-25T09:00:03Z"
  }
}
```

Consumers:

Consumer	Purpose
DD_NOT-01	Customer SMS+email with the paybill, account number, deadline. The most important customer notification of the journey.
CVM	Funnel stage tracking.
Channel UIs (PWA, Backoffice)	Display the payment reference if the channel is still viewing the order.

sophix.orders.payment-received

Emitted when cumulative $\geq$ expected.

```
{
  "eventType": "OrderPaymentReceived",
  "aggregateId": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
  "timestamp": "2026-05-18T11:23:02.789Z",
  "payload": {
    "orderId": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
    "customerId": "cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6",
    "operatorCode": "WIK",
    "subscriptionId": "sub_01HX5M2K9P3Q7R4T6W8Y0Z2N4Q",
    "paymentId": "pmt_01HZ_MPESA_NK4XX7QP9",
    "paidAmount": 15000.00,
    "currency": "KES",
    "method": "MPESA",
    "cumulativePaid": 15000.00,
    "expectedAmount": 15000.00,
    "receivedAt": "2026-05-18T11:23:01Z"
  }
}
```

sophix.orders.payment-received-partial

```
{
  "eventType": "OrderPaymentReceivedPartial",
  "aggregateId": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
  "timestamp": "2026-05-18T11:23:02.789Z",
  "payload": {
    "orderId": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
    "customerId": "cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6",
    "operatorCode": "WIK",
    "paymentId": "pmt_01HZ_PARTIAL_1",
    "paidAmount": 10000.00,
    "currency": "KES",
    "method": "MPESA",
    "cumulativePaid": 10000.00,
    "expectedAmount": 15000.00,
    "shortfallAmount": 5000.00,
    "receivedAt": "2026-05-18T11:23:01Z"
  }
}
```

DD_NOT-01 → "received X, please pay remaining Y."

sophix.orders.compute-payment-warning

```
{
  "eventType": "OrderComputePaymentWarning",
```

```

    "aggregateId": "ord_01HZ...COMP",
    "timestamp": "2026-05-18T09:00:03.234Z",
    "payload": {
      "orderId": "ord_01HZ...COMP",
      "operatorCode": "WIK",
      "packageRef": "pkg_COMP_TEST",
      "warning": "ZERO_COST_ACTIVATION",
      "detail": "No ACTIVATION_INTENT BillableEvents fire for this Package; expected_payment_amount = 0"
    }
  }
}

```

Consumed by ops/NOC for catalog verification.

9. Step-pinned rules

ID	Rule
R-FUL-02-PAY-1	expected_payment_amount is a snapshot computed at order-compute time. Not re-evaluated on partial-payment cycles. The actual charge during activation (BIL-01 against the customer's credit balance / wallet) may differ slightly due to operator overflow policy / discounts re-applied; differences are reconciled by BIL-01.
R-FUL-02-PAY-2	PaymentAppliedConsumer correlates via customer_order_pending_payments.payment_reference = event.targetReference (the customer typed order_id as the M-Pesa account number). The side-channel mapping table is written by order-notify-payment-instructions so the consumer has an explicit targetReference → order_id lookup independent of any invoice or subscription artifact (at payment time no invoice has been issued yet — payment lands on credit balance).
R-FUL-02-PAY-3	Partial-payment threshold: cumulative payment_received_amount ≥ expected_payment_amount → correlate PaymentReceived and resolve the pending-payments row. Below → emit payment-received-partial and keep parking.
R-FUL-02-PAY-4	Payment timeout duration is BPMN-defined: KE WIK ships PT7D. On boundary timer fire → BPMN routes via interrupting boundary event to STEP-CANCELLATION's order-cascade-cancellation with reason PAYMENT_TIMEOUT. Timer is NOT reset by partial payments — its absolute deadline applies.
R-FUL-02-PAY-5	At-least-once Kafka delivery requires idempotency: customer_order_processed_payments UNIQUE on (order_id, payment_id). Re-processing of the same payment_id is skipped silently.
R-FUL-02-PAY-6	Admin manual offline payment via POST /api/admin/orders/{id}/mark-paid-offline (BIL-01 has its own admin endpoint for this; we route through BIL-01) routes through normal POST /api/payments/receive flow → emits PaymentApplied → our consumer correlates PaymentReceived. No direct /engine-rest/message bypass.
R-FUL-02-PAY-7	When PaymentAppliedConsumer cannot find a matching pending-payments row (consumer arrived after timer fired, order is CANCELLED): forward original event to sophix.orders.message-correlation-dlq. Finance reviews — may issue refund via BIL-02-ADJ-01.
R-FUL-02-PAY-8	Currency derived from operator default (WIK→KES, WUG→UGX, WTZ→TZS, YASSN→XOF). Cross-currency payment (customer pays in different currency than the order's expected_payment_currency) is rejected by BIL-01 upstream of FUL-02; we never see it.
R-FUL-02-PAY-9	Zero-cost activation (no ACTIVATION_INTENT BillableEvents match the package) → expected_payment_amount = 0, partial-payment threshold check passes immediately, BPMN advances without waiting. Warning event emitted for ops verification (catalog inconsistency vs intentional free package).